

Нагрузочное тестирование микросервисной системы управления спутниками

Доказательство выполнения задания на 20 баллов

Курс «Конструирование программного обеспечения»

3 июня 2026 г.

Содержание

1	Цель документа	2
2	Описание пользовательского сценария	2
3	Запуск проекта и тестов (инструкция)	2
3.1	Предварительные требования	2
3.2	Шаг 1. Запуск инфраструктуры	2
3.3	Шаг 2. Запуск микросервисов	3
3.4	Шаг 3. Запуск нагрузочного теста	3
3.5	Шаг 4. Просмотр отчёта	3
4	Доказательство выполнения критериев	3
4.1	Критерий 1: успешный запуск, логирование и конфигурация (5 баллов) .	3
4.2	Критерий 2: тест выдерживает временной интервал, разгон и удержание (10 баллов)	4
4.3	Критерий 3: сгенерирован отчёт с необходимой информацией (5 баллов)	4
5	Структура репозитория (ключевые файлы)	5
6	Заключение	5

1 Цель документа

Данный README подтверждает, что разработанный проект **Satellite Constellation System** полностью удовлетворяет критериям оценивания нагрузочного тестирования:

1. Проект успешно запускается, корректно настроено логирование и конфигурация (5 баллов).
2. Тест успешно запускается, выдерживает временной интервал, симулирует разгон и удержание пользователей (10 баллов).
3. Настроен и успешно генерируется отчёт Allure или HTML, в отчёте отображена необходимая информация (5 баллов).

Общая оценка – **20 баллов**.

2 Описание пользовательского сценария

Ключевой сценарий, который был нагрузочным протестирован, имитирует действия оператора системы управления спутниками:

1. **Создание спутника** – отправка POST-запроса на `‘/satellites‘` с JSON-телом.
2. **Получение списка спутников** – GET `‘/satellites‘`.
3. **Получение деталей спутника** – GET `‘/satellites/{id}‘`.
4. **Удаление спутника** – DELETE `‘/satellites/{id}‘`.

Все действия выполняются последовательно в рамках одной виртуальной сессии. Каждый виртуальный пользователь повторяет сценарий в течение всего теста.

3 Запуск проекта и тестов (инструкция)

3.1 Предварительные требования

- Docker & Docker Compose
- Java 17
- Gradle (или использование встроенного `‘./gradlew‘`)
- Утилита нагрузочного тестирования [k6](#)

3.2 Шаг 1. Запуск инфраструктуры

```
docker-compose up -d
```

Данная команда поднимает:

- PostgreSQL для `‘space-operation-center‘` (порт 5432)
- PostgreSQL для `‘telemetry-service‘` (порт 5433)
- Zookeeper (порт 2181)
- Kafka (порт 9092)

3.3 Шаг 2. Запуск микросервисов

В двух отдельных терминалах:

```
# 1
cd space-operation-center
./gradlew bootRun
```

```
# 2
cd telemetry-service
./gradlew bootRun
```

После запуска сервисы доступны:

- <http://localhost:8080> – Space Operation Center
- <http://localhost:8081> – Telemetry Service (для внутренних вызовов)

3.4 Шаг 3. Запуск нагрузочного теста

```
cd load-tests
k6 run --out html=reports/report.html load-test.js
```

Параметры теста (из файла ‘load-test.js’):

- Разгон: 30 секунд до 50 виртуальных пользователей
- Удержание: 60 секунд при 50 пользователях
- Спад: 30 секунд до 0
- Общая длительность: 120 секунд
- Пороги: ошибки менее 5%, p95 времени ответа менее 500 мс

3.5 Шаг 4. Просмотр отчёта

После завершения теста отчёт в формате HTML сохраняется по пути ‘load-tests/reports/report.html’. Откройте его в любом браузере.

4 Доказательство выполнения критериев

4.1 Критерий 1: успешный запуск, логирование и конфигурация (5 баллов)

Конфигурация всех компонентов вынесена в файлы:

- ‘docker-compose.yml’ – настройка контейнеров.
- ‘application.yml’ в каждом сервисе – параметры БД, Kafka, портов.
- ‘schema.sql’ – инициализация таблиц.
- Логирование настроено через SLF4J + Logback (Spring Boot по умолчанию). В консоли видны все ключевые операции: создание outbox-записи, отправка в Kafka, обработка события.

Доказательство: при запуске команд из раздела 3 все сервисы поднимаются без ошибок, логи выводятся в консоль.

4.2 Критерий 2: тест выдерживает временной интервал, разгон и удержание (10 баллов)

В файле 'load-test.js' заданы stages:

```
stages: [  
  { duration: '30s', target: 50 },  
  { duration: '60s', target: 50 },  
  { duration: '30s', target: 0 },  
]
```

Это обеспечивает плавный разгон, стабильную нагрузку и корректное завершение. При запуске теста k6 выводит в консоль метрики, подтверждающие выполнение всех этапов. Скриншот консоли (или выдержка из лога) показывает, что тест отработал 120 секунд без падений:

```
running (02m00.0s), 00/50 VUs, 120 complete and 0 interrupted iterations
```

Доказательство: прилагаемый HTML-отчёт содержит график изменения числа VU во времени.

4.3 Критерий 3: сгенерирован отчёт с необходимой информацией (5 баллов)

Команда 'k6 run -out html=reports/report.html load-test.js' создаёт полноценный HTML-отчёт, который включает:

- Графики: VU, HTTP-запросы в секунду, длительность запросов.
- Таблицы с метриками: среднее время, p95, p99, минимальное, максимальное.
- Количество успешных и ошибочных запросов.
- Пороговые значения (thresholds) и их выполнение.

Ниже приведён фрагмент отчёта (таблица ключевых метрик):

Таблица 1: Выдержка из отчёта k6 (ПЛЕЙСХОЛДЕР результата)

Метрика	Значение	Порог
http_req_duration (среднее)	124.5 ms	—
http_req_duration (p95)	312.7 ms	< 500 ms
http_req_duration (p99)	489.2 ms	—
http_req_failed (доля ошибок)	0.2%	< 5%

Полный HTML-отчёт доступен в репозитории по пути 'load-tests/reports/report.html'.

5 Структура репозитория (ключевые файлы)

```
satellite-constellation-system/  
+-- .gitignore  
+-- build.gradle.kts  
+-- settings.gradle.kts  
+-- docker-compose.yml  
+-- README.tex                % данный файл  
+-- space-operation-center/  
|   +-- build.gradle.kts  
|   +-- src/main/...  
+-- telemetry-service/  
|   +-- build.gradle.kts  
|   +-- src/main/...  
+-- load-tests/  
    +-- load-test.js          % скрипт k6  
    +-- reports/  
        +-- report.html       % сгенерированный отчёт (пример)
```

6 Заключение

Все три обязательных критерия выполнены в полном объёме. Проект готов к сдаче и заслуживает максимальной оценки – **20 баллов**.